TP 3 : Many lights, culling, clustering et éclairage différé

Frustum culling, clustering du frustum, passe d'éclairage en compute

Programmation parallèle GPU : OpenGL 4.3 / GLSL 4.30

Objectifs

— éliminer les lumières hors du champ de vision par **frustum culling** de leurs sphères d'influence ;

— découper le volume de vue en **clusters** (*clustered shading*) et construire, en compute shader, la liste des lumières de chaque cluster ;

— éclairer la scène par une **passe d'éclairage en compute** : G-buffer → une invocation par pixel → image → affichage ;

— valider l'image obtenue contre une référence exhaustive et mesurer le gain.

Prérequis

TP1 et TP2 (SSBO, dispatchs multiples, barrières, atomiques, listes). Notions de rendu : matrice vue-projection, repères monde/caméra, lumière ponctuelle.

1 Environnement et éléments fournis

Le dossier `squelette/` (C++11, gcc, CMake, cf. TP1) fournit le contexte OpenGL 4.3, la compilation des shaders, les utilitaires de buffers du TP1, ainsi que :

— une scène test avec 1 000 à 4 000 lumières ponctuelles animées ;

— la passe de rendu **G-buffer** : positions, normales et albédos sont rendus dans trois textures (l'objectif du TP est l'éclairage, pas la construction du G-buffer) ;

— l'affichage plein écran d'une texture ;

— les fonctions ci-dessous. Les passes compute restent à écrire.

Interface du squelette

```
// Rendu du G-buffer dans les textures attachees
// (positions, normales, albedo), plus la profondeur view-space.
void renderGBuffer();

// Affichage plein ecran d'une texture 2D.
void displayTexture(GLuint texture);

// Matrices courantes de la camera.
const glm::mat4& viewMatrix();
const glm::mat4& projMatrix();
float zNear();
float zFar();
float positiveDepthFromWindowZ(float windowZ);

// Lumieres animes (monde). Mettre a jour une fois par image.
const std::vector<PointLight>& lights();
void updateLights(float timeSeconds);
```

2 Conventions et représentation des données

2.1 Conventions caméra et profondeur

Toutes les positions utilisées pour les tests spatiaux sont exprimées dans le **repère caméra** (*view space*), l'axe $-z$ pointant devant la caméra. On note z_{near} et z_{far} les plans proche et lointain, et

$$d(\mathbf{p}) = -p_z \ > 0$$

la **profondeur positive** d'un point $\mathbf{p}$ en repère caméra. La correspondance entre la coordonnée z de la fenêtre et d est positiveDepthFromWindowZ (et `shaders/depth.glsl`) ; elle doit être *identique* dans toutes les passes.

2.2 Lumières

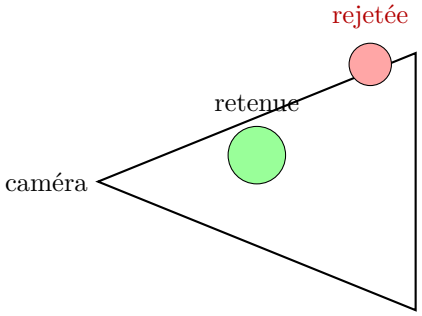
```
struct PointLight {
    vec4 positionRadius; // xyz : position (repere MONDE), w : rayon
    vec4 colorIntensity; // rgb : couleur, w : intensite
};
```

Chaque lumière possède une influence **strictement nulle au-delà de son rayon** : le test d'influence est $d_{light} < r_{light}$. Les positions sont converties en repère caméra *avant* les tests, dans une passe de préparation (extension : directement dans le shader de culling).

Attention

Une lumière dont le *centre* est hors du frustum peut néanmoins éclairer la scène visible : son rayon déborde. Le culling doit donc tester la **sphère d'influence**, pas le point.

3 Exercice 1 : Frustum culling des lumières



test de la sphère contre les six plans

3.1 Extraction des plans

Les lumières étant testées en repère caméra, le frustum est ici celui de la seule **matrice de projection** P. Chaque plan s'écrit $\mathbf{n}\cdot\mathbf{x}+b=0$, les six plans étant : gauche, droite, bas, haut, proche, lointain. On les obtient en combinant les lignes $r_1,\dots,r_4$ de P (méthode de Gribb–Hartmann), puis en normalisant l'*équation entière* par la norme de sa normale :

gauche :	$r_4 + r_1$	(composante à composante)
droite :	$r_4 - r_1$	
bas :	$r_4 + r_2$	
haut :	$r_4 - r_2$	
proche :	$r_4 + r_3$	
lointain :	$r_4 - r_3$	

(convention : l'intérieur est $\mathbf{n}\cdot\mathbf{x}+b\geq 0$).

3.2 Test sphère / plan

Pour une sphère de centre $\mathbf{c}$ et rayon r :

$$\delta = \mathbf{n}\cdot\mathbf{c}+b \qquad \begin{cases} \delta < -r & \Rightarrow \text{sphère entièrement hors du plan} \\ \delta \geq -r & \Rightarrow \text{conservée} \end{cases} \tag{1}$$

La lumière est visible si sa sphère n'est *pas* entièrement rejetée par au moins l'un des six plans.

3.3 Shader de culling

Une invocation par lumière :

```
#version 430 core
layout(local_size_x = 256) in;

layout(std430, binding = 0) readonly buffer Lights {
    PointLight lights[];
};
layout(std430, binding = 1) buffer Vis {
    uint visibleCount; // compte les lumieres retenues
    uint visible[];    // indices des lumieres retenues
};

uniform uint numLights;
uniform vec4 planes[6]; // n.xyz, c : plans du frustum

void main() {
```

```
    // TODO : test sphere / 6 plans, insertion dans visible[]
}
```

1. Écrire le test et l'insertion avec atomicAdd sur visibleCount. Comment garantir qu'on n'écrit jamais au-delà de numLights ?
2. Le compteur visibleCount et le tableau visible [] doivent être **réinitialisés** avant chaque image (dispatch séparé ou clearSSBO, puis barrière SSBO). Pourquoi ?
3. Pourquoi la passe de culling est-elle un dispatch séparé de la passe de clustering, alors que les deux lisent les mêmes lumières ?
4. Quelle barrière sépare le culling du clustering ?

3.4 Validation

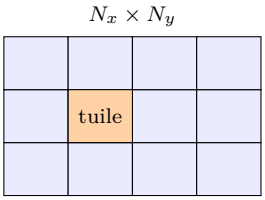
Implémenter la même élimination côté CPU (test identique) et vérifier :

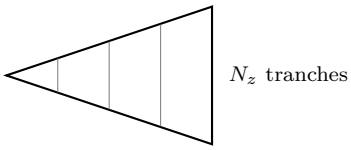
— mêmes lumières retenues, à l'ordre près ;

— cas limites : lumière centrée derrière la caméra mais de grand rayon (doit être conservée) ; lumière tangent au plan proche ;

— aucune lumière visible rejetée (faux négatif) : c'est le critère de correction qui compte, pas le nombre de rejets.

4 Exercice 2 : Clustering du frustum





4.1 Découpage

Le volume de vue est découpé en une grille 3D alignée caméra :

— $N_x \times N_y$ découpes écran de taille fixe (par exemple des tuiles de 64×64 pixels, calculées depuis la résolution) ;

— N_z **tranches linéaires** en profondeur : la tranche k couvre $d \in [z_{near}+k\,\Delta z,\ z_{near}+(k+1)\,\Delta z]$ avec $\Delta z = (z_{far} - z_{near})/N_z$.

Le **cluster** d'un pixel de coordonnées fenêtre (x, y) et de profondeur d est :

$$cluster(x, y, d) = \left(\left\lfloor \frac{x}{s_x} \right\rfloor, \left\lfloor \frac{y}{s_y} \right\rfloor, \min\left(N_z - 1,\ \left\lfloor \frac{d - z_{near}}{\Delta z} \right\rfloor \right) \right). \tag{2}$$

Attention

Contrat essentiel : la fonction qui calcule le cluster d'un fragment (passe de rendu) et celle qui construit les volumes englobants des clusters (passe de construction) doivent être **la même fonction**. Toute divergence (bornes, arrondis, N_z) produit des lumières manquantes dans l'image. Factorisez-la dans un fichier inclus par les deux shaders.

4.2 Volumes englobants des clusters

Chaque cluster est la portion du frustum d'une tuile écran, entre deux plans de profondeur. En repère caméra, sa boîte englobante (AABB) s'obtient en **déprojetant les huit coins** de la tuile (les quatre coins écran de la tuile, à chacune des deux profondeurs d_{min} et d_{max} de la tranche), puis en prenant le minimum et le maximum par axe. En perspective, la section d'un cluster *n'est pas constante* en profondeur : la déprojection est la méthode sûre, à préférer à toute approximation par dérivées partielles.

1. Écrire la fonction de déprojection d'un point écran (x, y) à profondeur d donnée (inverse de la projection).
2. Construire, pour chaque cluster, son AABB en repère caméra, dans un SSBO (un seul tableau de structures, placé en dernier) :

```
struct AABB {
    vec4 mn;  // xyz : coin min, w : inutilise
    vec4 mx;  // xyz : coin max, w : inutilise
};

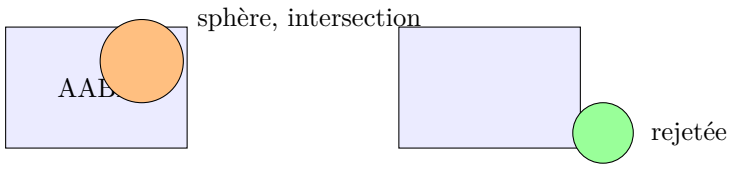
layout(std430, binding = 2) buffer ClusterAABB {
    AABB aabb[];
};
```

3. Vérifier, côté CPU, que l'union des AABB couvre le frustum et qu'aucun point visible n'est hors de tout cluster.

4.3 Question de conception

Pourquoi des tranches *linéaires* sont-elles suffisantes pour l'objectif du TP, et que changerait un découpage *logarithmique* (densité constante de clusters par unité relative de profondeur) ? L'implémentation des tranches logarithmiques est une extension, pas un objectif.

5 Exercice 3 : Affectation lumière → cluster



5.1 Principe

Pour chaque cluster, on retient les lumières dont la sphère d'influence **peut** intersecter le cluster. Le test est **conservatif** : il peut retenir une lumière en trop, mais ne doit jamais en oublier une qui éclaire réellement le cluster. On teste l'intersection sphère/AABB (distance au carré du centre de la sphère à la boîte, à comparer à r^2).

```
// Distance carree entre ‘c‘ et la boite [mn, mx]
float distSqAABB(vec3 c, vec3 mn, vec3 mx) {
    vec3 q = max(mn - c, max(c - mx, vec3(0.0)));
    return dot(q, q);
}
// Intersection si distSqAABB(center, mn, mx) <= r * r
```

5.2 Shader d'affectation

Une invocation par couple (cluster, lumière) : la grille de dispatch couvre numClusters × numLights, borne supérieure lisible côté CPU *sans* attendre visibleCount :

```
#version 430 core
layout(local_size_x = 64, local_size_y = 8) in;
// x : indice de cluster, y : indice de lumiere

layout(std430, binding = 1) readonly buffer Vis {
    uint visibleCount;
    uint visible[];
};
layout(std430, binding = 2) readonly buffer ClusterAABB {
    AABB aabb[];
};
// Trois SSBO separes : un bloc ne contient qu'un seul
// tableau de taille indeterminee, en dernier.
layout(std430, binding = 3) buffer LightCount {
    uint lightCount[]; // par cluster
};
layout(std430, binding = 4) buffer LightIndices {
    uint lightIndices[]; // cluster * MAX_PER_CLUSTER + slot
};
layout(std430, binding = 5) buffer Overflow {
    uint overflow[];   // drapeau par cluster
};

uniform uint numClusters;
uniform uint numLights;
uniform uint maxPerCluster;

void main() {
    uint c = gl_GlobalInvocationID.x; // cluster
    uint v = gl_GlobalInvocationID.y; // indice dans visible[]
    if (c >= numClusters || v >= visibleCount) return;
```

```
    uint lightIndex = visible[v];
    // TODO : test sphere/AABB, insertion dans la liste du cluster
}
```

Avant ce dispatch, réinitialiser lightCount et overflow : dispatch séparé ou clearSSBO, puis barrière SSBO.

Insertion :

```
uint slot = atomicAdd(lightCount[c], 1u);
if (slot < maxPerCluster)
    lightIndices[c * maxPerCluster + slot] = lightIndex;
else
    atomicMax(overflow[c], 1u);
```

À la lecture, la boucle est toujours bornée par la capacité allouée :

```
uint count = min(lightCount[c], maxPerCluster);
```

sinon le compteur, qui a continué à croître au-delà de la capacité, ferait lire hors de la zone attribuée au cluster.

5.3 Questions

1. Pourquoi le test conservatif suffit-il à la correction de l'image finale, alors qu'il retient des lumières en trop ?
2. Pourquoi une lumière peut-elle appartenir à **plusieurs** clusters, alors qu'une particule du TP2 n'appartenait qu'à une seule cellule ?
3. Que se passe-t-il si plusieurs invocations insèrent dans la même liste ? Pourquoi l'ordre des indices dans une liste est-il non défini, et pourquoi cela n'a-t-il aucune importance ?
4. Le drapeau overflow doit être **visible dans l'image de diagnostic**, jamais masqué : pourquoi refuse-t-on la troncature silencieuse ?

5.4 Gestion des dépassements

Pour les images de validation, choisir maxPerCluster suffisant (par exemple 256), et afficher en rouge les clusters en dépassement. En extension : repli vers le parcours exhaustif des lumières pour les seuls clusters en dépassement.

5.5 Validation

— Pour chaque cluster, comparer la liste GPU à un calcul CPU du même test conservatif : mêmes ensembles d'indices, à l'ordre près ;

— vérifier qu'**aucune** lumière qui intersecte un cluster n'est absente de sa liste (c'est la correction exigée) ;

— compter le nombre moyen et maximal de lumières par cluster.

6 Exercice 4 : Passe d'éclairage en compute



6.1 Modèle d'éclairage

Pour un point de surface $\mathbf{p}$, de normale $\mathbf{n}$ et d'albédo ρ, éclairé par une lumière $(\mathbf{l}, r, \mathbf{c}, I)$:

$$d = \|\mathbf{l} - \mathbf{p}\|, \qquad L_o \mathrel{+}= \rho \odot \mathbf{c}\, I\, A(d)\ \max(0, \mathbf{n} \cdot \hat{\mathbf{l}}_p), \tag{3}$$

avec l'atténuation bornée :

$$A(d) = \begin{cases} (1 - d/r)^2 & d < r, \\ 0 & d \geq r, \end{cases} \tag{4}$$

et $\hat{\mathbf{l}}_p$ la direction de $\mathbf{p}$ vers $\mathbf{l}$.

6.2 Référence exhaustive

Avant tout clustering, écrire la version naïve : pour chaque pixel, parcourir **toutes les lumières de la scène**, sans culling ni clustering. Cette version sert de référence de correction *et* de performances : si une lumière était rejetée à tort par le culling, une version « exhaustive » qui s'appuierait sur le culling cacherait l'erreur dans les deux images. L'image finale du TP doit être *identique* à cette référence à une tolérance flottante près.

6.3 Passe compute

Une invocation par pixel :

1. lire le G-buffer (position, normale, albédo) à ses coordonnées ;
2. calculer le cluster du pixel avec la *même fonction* que la construction ;
3. parcourir la liste du cluster, appliquer le modèle d'éclairage avec le **test de rayon exact** ;
4. écrire la couleur dans une image de sortie (déclaration dans le listing ci-dessous).

```
layout(std430, binding = 4) readonly buffer LightCount {
    uint lightCount[];
};
layout(std430, binding = 5) readonly buffer LightIndices {
    uint lightIndices[];
};
layout(std430, binding = 6) readonly buffer Lights {
    PointLight lights[];
};

layout(rgba8, binding = 0) writeonly uniform image2D outputImage;

// TODO : G-buffer en textures, cluster(x, y, d), boucle, eclairage
```

Côté hôte :

```
glDispatchCompute(ceilDiv(width, 16), ceilDiv(height, 16), 1);
glMemoryBarrier(GL_TEXTURE_FETCH_BARRIER_BIT);
displayTexture(outputTexture);
```

1. Justifier la barrière choisie : qui lit l'image, et comment ?
2. Pourquoi la taille locale 16×16 convient-elle bien ici, alors que les tailles de l'exercice 1 étaient 1D ?

6.4 Enchaînement complet d'une image

1 rendu du G-buffer (fourni)
2 dispatch culling des lumières (une invocation par lumière)
3 barrière SSBO
4 dispatch construction des listes (une invocation par (cluster, lumière visible))
5 barrière SSBO
6 dispatch éclairage (une invocation par pixel)
7 barrière texture / fetch
8 affichage de l'image

6.5 Validation et diagnostics

— Image clusterisée vs exhaustive : différence maximale par canal tolérée à justifier (arrondi flottant, ordre des additions) ;

— vues de diagnostic commutables : identifiant de cluster (couleur), nombre de lumières par cluster, clusters en dépassement, différence avec la référence ;

— déplacer la caméra : vérifier qu'aucun scintillement n'apparaît (symptôme d'une divergence entre construction et recherche de cluster).

6.6 Mesures et analyse

1. Mesurer séparément : culling, construction des listes, passe d'éclairage, total (moyenne sur 300 images, après warm-up) ;
2. comparer à la référence exhaustive (même passe d'éclairage, sans les listes) pour 500, 1 000 et 4 000 lumières ;
3. faire varier la taille des tuiles (32, 64, 128 pixels) et N_z (8, 16, 32) : mesurer l'effet sur la mémoire des listes et le nombre moyen de lumières par cluster ;
4. dans quelles configurations (lumières lointaines ou très étalées) le coût de construction dépasse-t-il le gain au rendu ?

7 Extensions

1. **Listes compactes** : remplacer les zones de capacité fixe par clusterOffset / clusterCount / liste globale d'indices, construites en trois passes (comptage, somme préfixe, remplissage). La somme préfixe (*scan*) est une primitive à part entière : commencer par une version multi-dispatch naïve.
2. **Tranches logarithmiques** en profondeur.
3. **Repli exhaustif** pour les clusters en dépassement.

8 Livrables

Les quatre passes (culling, AABB des clusters, affectation, éclairage) avec leurs shaders et code hôte ; la référence exhaustive ; la fonction de cluster factorisée et partagée ; les vues de diagnostic ; les tests de correction (aucune lumière manquante, image identique à la référence) ; les mesures séparées ; l'analyse des cas défavorables. Les réponses aux questions numérotées figurent dans le compte rendu.